M4010104 / M4020104: PROGRAMMING IN PYTHON · DIGITAL UNIVERSITY KERALA · CAPSTONE PROJECT SPECIFICATION

Project 10: Aksharantar Manglish-to-Malayalam Transliteration & Phonetic Alignment Profiler

Effort Budget: 10–12 Hours | Core Stack: Pure Python, NumPy, Pandas, Matplotlib, Seaborn, Plotly | Max Marks: 25 | Instructor: Dr. Sunil TT

1. Executive Overview & Domain Context

Millions of Malayalam speakers communicate on social media platforms using the Roman alphabet ('Manglish'). The AI4Bharat Aksharantar benchmark provides large-scale mined pairs of Romanized text mapped to native Malayalam Unicode words, enabling systematic investigation of phonetic mapping ambiguity, vowel elongation, and retroflex consonant orthography.

Your team will architect an empirical script alignment and transliteration profiling tool in pure Python that cleans bilingual word pairs, calculates Roman-to-Malayalam character expansion ratios, constructs character n-gram Pointwise Mutual Information (PMI) transition matrices in NumPy, and visualizes mapping ambiguities answering the 12 core questions below.

Primary Public Dataset & Telemetry/Data Schema

The project utilizes the verified public AI4Bharat Aksharantar (Malayalam Subset) on Hugging Face (`https://huggingface.co/datasets/ai4bharat/Aksharantar`). The dataset structure captures:

- **Raw Feature Attributes:** `english_roman` (Manglish word), `malayalam_native` (Unicode Malayalam word).
- **Engineered / Target Variables:** `roman_len`, `native_len`, `expansion_ratio`, `syllable_count`, `pmi_mapping_score`, `has_chillaksharam`.

2. Complete Project Directory & File Architecture

Each team must organize their repository into a standardized, production-grade layout. All application source code must reside inside a dedicated package (`src/`), with separate directories for raw/curated data, interactive development notebooks, visual figures, and formal documentation:

```
manglish-malayalam-transliteration-profiler/
├── README.md                          # Repository documentation & GitHub Pages source
├── requirements.txt                   # Dependency specification (numpy, pandas, matplotlib, seaborn, p
├── run_pipeline.py                    # Standalone production CLI application entry point
├── data/
│   ├── raw/
│   │   └── aksharantar_malayalam_raw.csv            # Raw uncleaned input dataset (including simulated/
│   └── processed/
│       ├── clean_aksharantar_ml.csv          # Curated, validated clean CSV dataset
│       └── transliteration_audit.log           # Timestamped audit report detailing filtered/rejected r
├── src/
│   ├── __init__.py                    # Python package initialization
│   ├── ingestion.py                   # Defensive line-by-line file reader, range auditor & CSV exporte
│   ├── analyzer.py                    # TransliterationProfiler OOP class, NumPy vectorization & Pandas
│   └── visualizer.py                  # Matplotlib envelope plots, Seaborn heatmaps & Plotly HTML expor
├── notebooks/
│   └── development_cycle.ipynb        # Development cycle notebook: EDA, trials, tests & unit assertion
└── reports/
    ├── figures/
    │   ├── primary_envelope.png       # Matplotlib confidence envelope & regression overlay
    │   ├── correlation_heatmap.png    # Seaborn annotated feature correlation heatmap
    │   └── distribution_regimes.png   # Seaborn distribution across categorical/domain tiers
```

```
├── interactive_dashboard.html     # Standalone interactive Plotly HTML dashboard with controls
└── Project_Report.pdf             # 3-4 page final technical report
```

3. Dual Development Deliverables: Notebook vs. Standalone Program

To mirror industry software engineering standards, each student group must deliver two distinct, complementary implementations of their solution:

Deliverable A: The Development Cycle Notebook (`notebooks/development_cycle.ipynb`)

The development notebook chronicles your team's engineering journey from raw data exploration to validated model prototypes. It is not merely a final script pasted into cells, but an authentic record of the development cycle:

- **Phase 1: Exploratory Data Ingestion & Diagnostics:** Loading raw data, inspecting missing values, testing boundary rules, and logging rejected records.
- **Phase 2: Mathematical Formulation & Feature Verification:** Developing and validating NumPy array calculations against target benchmarks using `np.allclose()`.
- **Phase 3: OOP Class Architecture & Unit Tests:** Instantiating the `TransliterationProfiler` class, executing automated unit assertions (e.g., verifying that passing invalid parameters raises an explicit `ValueError`), and testing edge cases.
- **Phase 4: Statistical Modeling & Regimen Exploration:** Implementing ordinary least-squares regression, rolling-window calculations, and multi-metric Pandas grouping.
- **Phase 5: Visual Prototype Development:** Iteratively designing and refining Matplotlib and Seaborn figures, followed by configuring the interactive Plotly HTML dashboard.

Deliverable B: The Final Standalone Python Program (`run_pipeline.py` & `src/`)

While the notebook documents the exploratory development cycle, your repository must also provide a production-ready, fully executable Python application. The program must run end-to-end directly from the terminal without requiring Jupyter:

- **Command-Line Interface (CLI):** Executed via `python run_pipeline.py --input data/raw/aksharantar_malayalam_raw.csv --output-dir data/processed --report-dir reports`.
- **Modular Package Import:** The driver script imports cleanly separated domain logic from `src.ingestion`, `src.analyzer`, and `src.visualizer`.
- **Terminal Feedback & Exit Codes:** The program must print structured progress indicators, log clean/faulty record counts to the console, and return an exit code of `0` upon successful pipeline completion.
- **Automated Artifact Generation:** In a single execution pass, the script reads raw inputs, writes `data/processed/clean_aksharantar_ml.csv`, saves `data/processed/transliteration_audit.log`, exports static figures to `reports/figures/`, and generates the standalone `reports/interactive_dashboard.html`.

4. Technical Investigation Requirements & Research Questions (Q1–Q12)

Your team's implementation across the development notebook and standalone program must rigorously answer the following 12 course-aligned engineering questions:

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
File I/O & Exceptions (`src/ingestion.py`)	Q1	**Bilingual Word Pair Integrity & Script Audit:** Parse raw word pairs line-by-line with try-except. Validate that Roman words contain strictly ASCII letters ([a-zA-Z]) and target words contain valid Malayalam Unicode code points.	open(), try-except (ValueError, IndexError), regex validation

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
Data Curation & Export (`src/ingestion.py`)	**Q2**	**Curated Transliteration Corpus Export & Audit:** Export verified word pairs to clean_aksharantar_ml.csv while logging non-matching script rows, whitespace errors, and punctuation anomalies in transliteration_audit.log.	Defensive file writing, dictionary parsing, formatted log reporting
OOP Design (`src/ingestion.py`)	**Q3**	**Transliteration Profiler Class:** Design an encapsulated class TransliterationProfiler with methods load_from_csv(), compute_pmi_matrix(min_cooccurrence=5), and summary_report(). Ensure min_cooccurrence < 1 raises ValueError.	OOP class, __init__, encapsulation, raise ValueError
NumPy Arrays (`src/analyzer.py`)	**Q4**	**Vectorized Expansion Ratios & Verification:** Using only raw NumPy character length arrays, compute character expansion ratios (Native_Len / Roman_Len). Verify distribution statistics via np.allclose().	np.divide(), array broadcasting, np.allclose()
NumPy Masking (`src/analyzer.py`)	**Q5**	**High-Ambiguity Phonetic Digraph Masking:** Flag Roman digraphs (e.g., 'zh', 'th', 'ch', 'ng') exhibiting high transliteration variance in NumPy. Identify whether 'zh' maps consistently to Malayalam retroflex \u0D34 (ZHA) via set operations.	np.mean(), np.std(), boolean masks (&, |), Python set
NumPy Regression (`src/analyzer.py`)	**Q6**	**Roman Length vs. Malayalam Length OLS Model:** Fit an ordinary least-squares model (Native_Len = b0 + b1*Roman_Len) via np.linalg.lstsq. Quantify the slope of character-length expansion and residual variance.	np.column_stack(), np.linalg.lstsq(), matrix product (@), R2
Pandas Series (`src/analyzer.py`)	**Q7**	**Word Length Distribution & Syllable Audit:** Compute average word lengths and character-to-syllable ratios across frequency tiers in Pandas. Audit the frequency of words containing chillaksharams (\u0D7A to \u0D7F).	.describe(), .apply(), .value_counts()
Pandas Groupby (`src/analyzer.py`)	**Q8**	**Word Length Discretization & Pivot Table:** Discretize Roman word lengths into Short (1-4), Medium (5-8), and Long (> 8 chars). Build a 3×2 pivot_table(margins=True) showing expansion ratios for chillaksharam vs. non-chillaksharam words.	pd.cut(), .groupby().agg(), .pivot_table(margins=True)
Pandas Reshaping (`src/analyzer.py`)	**Q9**	**Wide-to-Long Digraph Mapping Probability Ranking:** Reshape top Roman-to-Malayalam character co-occurrence frequencies into long format via pd.melt() and extract the top 5 most frequent target mappings per Roman digraph.	pd.melt(), .groupby('Roman_Digraph'), .nlargest(5)
Matplotlib (`src/visualizer.py`)	**Q10**	**Matplotlib Roman vs. Native Length Scatter Plot:** Plot Roman character length vs. Malayalam character length in Matplotlib. Overlay the OLS trendline, expansion boundary lines, and custom legends.	plt.subplots(), ax.scatter(), ax.plot(), custom legends
Seaborn (`src/visualizer.py`)	**Q11**	**Seaborn Phonetic Transition Probability Heatmap:** Construct a 2D transition probability matrix between top 15 Roman consonant prefixes and top 15 Malayalam Unicode characters; render with sns.heatmap(annot=True).	sns.heatmap(), sns.despine(), font properties
Plotly HTML (`src/visualizer.py`)	**Q12**	**Interactive Transliteration Search Dashboard:** Build an interactive HTML dashboard (interactive_dashboard.html) displaying Manglish search with instant native transliteration lookups, length distributions, and sliders.	plotly.express.scatter(), custom hover tooltips, write_html()

5. Collaborative Git Workflow, GitHub Pages & README.md Specification

Best Practices: Collaborative Git Workflow & GitHub Pages Deployment from Day 1

From the very first day, the student team must adopt professional version control and documentation practices. On Day 1, one designated team member initializes the central public GitHub repository, establishes a clear branch protection rule preventing direct pushes to **`main`**, adds all teammates as registered **Contributors** (with write access), and grants read access to the Course Instructor (**`Dr. Sunil TT`**) and teaching assistants. Immediately thereafter, the team must activate **GitHub Pages** (via *Settings → Pages → Build and deployment: Deploy from a branch: main /docs or root*) so that project documentation, interactive architecture diagrams, and the live Plotly dashboard (**`reports/interactive_dashboard.html`**) are publicly accessible and continuously deployed throughout the development lifecycle. All feature development must follow a strict branching model (e.g., **`feature/ingestion`**, **`feature/analysis`**, **`feature/visualization`**): individual team members must write small, atomic, well-commented commits from their own personal GitHub accounts and merge them into **`main`** exclusively through formal **Pull Requests (PRs)** accompanied by peer review. Dumping entire codebases in a single, monolithic end-of-semester commit is strictly prohibited and will result in zero marks for individual Git contribution; maintaining an unbroken, transparent Git commit and PR audit trail is essential for passing the code ownership defense during the live viva-voce examination.

Repository README.md Specification Template

The root **`README.md`** serves as the primary technical landing page and must strictly conform to the following structural format:

````
# Project 10: Aksharantar Manglish-to-Malayalam Transliteration & Phonetic Alignment Profiler
[![Course](https://img.shields.io/badge/Course-Programming%20in%20Python-blue.svg)](https://duk.ac.in)
[![Python](https://img.shields.io/badge/Python-3.11%2B-brightgreen.svg)](https://python.org)
[![License](https://img.shields.io/badge/License-MIT-yellow.svg)](LICENSE)

## 1. Project Overview & Domain Context
- **Institution:** Digital University Kerala (MTech AI / MSc AI)
- **Course:** M4010104 / M4020104 – Programming in Python | **Instructor:** Dr. Sunil TT
- **Dataset:** AI4Bharat Aksharantar (Malayalam Subset) on Hugging Face (https://huggingface.co/datasets

## 2. System Architecture & Pipeline Design
[Provide an ASCII or Mermaid architecture flow diagram showing raw ingestion -> validation -> analyzer -

## 3. Installation & Environment Setup
```bash
git clone https://github.com/<your-team>/manglish-malayalam-transliteration-profiler.git
cd manglish-malayalam-transliteration-profiler
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

## 4. Execution Instructions
### A. Running the Standalone Application
```bash
python run_pipeline.py --input data/raw/aksharantar_malayalam_raw.csv --output-dir data/processed --repo
```
### B. Running the Development Notebook
```bash
jupyter notebook notebooks/development_cycle.ipynb
```

## 5. Key Analytical & Statistical Findings (Q1–Q12 Summary)
[Provide a concise markdown summary table detailing raw vs. clean counts, regression coefficients, anoma

## 6. Interactive Telemetry Dashboard
Explore the live interactive Plotly visualization hosted on GitHub Pages:
🔗 [Live Project Dashboard](https://<your-team>.github.io/manglish-malayalam-transliteration-profiler/in
````

```
## 7. Contributor Git Commit & Pull Request Log Table
| Contributor Name | GitHub Handle | Assigned Module / Tasks | Verified Commit Hash Range | Merged Pull
| :--- | :--- | :--- | :--- | :--- |
| [Student Full Name] | `@github_handle` | Modular Architecture (`src/ingestion.py`, `src/analyzer.py`,
```

6. Evaluation Rubric & Mandatory Academic Integrity Governance

Projects are evaluated on a 25-mark scale based on code quality, mathematical correctness, visualization depth, git collaboration discipline, and live technical defense:

Assessment Criterion	Basis of Evaluation & Deliverable Verification	Max Marks
1. Data Curation & File I/O Pipeline (Q1–Q2)	Robust line-by-line file reading with defensive **`try-except`** handling, physical/domain bounds validation, clean CSV output, and structured **`transliteration_audit.log`** reporting.	**5 Marks**
2. OOP, NumPy & Pandas Analytics (Q3–Q9)	Modular **`TransliterationProfiler`** class architecture, vectorized NumPy feature re-derivations (verified via **`np.allclose`**), regression modeling, and Pandas rolling/groupby/melt operations.	**6 Marks**
3. Multi-Library Visualizations (Q10–Q12)	Publication-quality Matplotlib confidence envelope/trend plot, Seaborn correlation heatmap and distribution violin plot, and standalone interactive Plotly HTML dashboard.	**5 Marks**
4. Software Packaging, GitHub & Documentation	Clean directory layout, comprehensive **`README.md`**, working GitHub Pages site, **`development_cycle.ipynb`** notebook, standalone **`run_pipeline.py`**, and balanced individual Git commits/PRs.	**4 Marks**
5. Live Presentation & Code Viva-Voce Defense	Technical walkthrough and individual live viva defense where each student must explain or modify lines of code in real time without external assistance.	**5 Marks**

> **STRICT ACADEMIC INTEGRITY & ZERO-TOLERANCE AI POLICY (0 MARKS PENALTY):** All submitted Python source code (**`.py`**, **`.ipynb`**) and written documentation will be screened through an automated **AI-Code & Plagiarism Detection Suite**. If AI-generated code or synthetic documentation is detected at any checkpoint or final submission, **0 Marks will be awarded immediately** for the entire project. Each student must prove authentic individual code ownership during the live viva-voce examination — *Course Instructor: Dr. Sunil TT*.

7. Milestone Schedule, Submission Checklist & Viva Defense Prompts

To ensure systematic progress across the semester, teams must adhere to three project checkpoints and prepare for on-the-spot live code modification challenges during the viva-voce examination:

Milestone Stage	Scheduled Window	Required Deliverables & Verification Milestones
Checkpoint 1 (Data Ingestion)	Week 11 Lab Session	Repository initialized with contributors and GitHub Pages active. **`src/ingestion.py`** operational with line-by-line filtering, **`clean_aksharantar_ml.csv`** exported, and **`transliteration_audit.log`** generated.
Checkpoint 2 (Analytics & OOP)	Week 13 Lab Session	**`src/analyzer.py`** complete with **`TransliterationProfiler`** class, NumPy feature checks (Q4), outlier masks (Q5), regression modeling (Q6), and exploratory notebook sections.

Milestone Stage	Scheduled Window	Required Deliverables & Verification Milestones
Final Submission & Viva	Week 15 Project Review	Full package operational: standalone **`run_pipeline.py`**, polished **`development_cycle.ipynb`**, interactive **`interactive_dashboard.html`** on GitHub Pages, 3–4 page PDF report, and live code defense.

Representative Live Viva-Voce Code Modification Challenge

During the individual code viva-voce, the instructor will ask students to execute an immediate 2-minute code modification on their working repository to demonstrate genuine authorship:

- **Challenge Prompt:** *"In src/analyzer.py, compute the top 5 Roman character digraphs mapping to chillaksharam 'L' (\u0D7D) vs. retroflex 'LL' (\u0D7E). Re-run your module and report the frequency counts."*